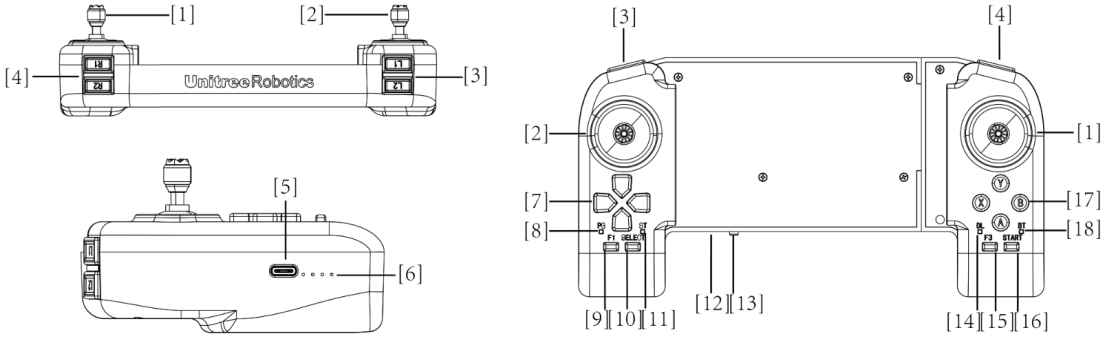
项目 6 遥控器状态获取

1、 预备知识

遥控器的各部分按键说明如图所示。遥控器的功能主要由左右摇杆和功能按键组成。每个摇杆均有两个自由度，能够输出 x 和 y 方向上的摇杆数值，其范围为 **[-1.0~1.0]**。功能按键/功能按键组合都对应一个唯一的整型键值。底层服务 rt/lowstate 中存储了一个 40 字节的遥控器原始数据，用户可以转化为遥控器数据结构后使用。



[1] 右摇杆 [2] 左摇杆 [3] L1/L2 键 [4] R1/R2 键 [5] 充电接口 Type C [6] 数传信号灯 [7] 左按键 [8] 电源连接指示灯 [9] F1 (左摇杆校准键) [10] SELECT 键 [11] 充电状态指示灯 [12] 电源指示灯 [13] 电源键 [14] 数据传输指示灯 [15] F3 (右摇杆校准键) [16] START 键 [17] 右按键 [18] 蓝牙信号指示灯

图 2-1-1 遥控器按键说明

2、 遥控器状态获取

2.1、 C++例程实现

2.1.1、 获取遥控器数据

```
unitree_go::msg::dds_::WirelessController_ joystick; //遥控器数据dds
joystick.lx(); //获取左摇杆 x 数据，范围[-1.0~1.0]
joystick.ly(); //获取左摇杆 y 数据，范围[-1.0~1.0]
joystick.rx(); //获取右摇杆 x 数据，范围[-1.0~1.0]
joystick.ry(); //获取右摇杆 y 数据，范围[-1.0~1.0]
joystick.keys(); //获取遥控器键值
```

遥控器键值和按钮之间的对应关系由以下联合体定义。将获取到的键值赋给联合体中的 value 变量，再通过判断 components 中的成员变量是否为 1，可判断对应的按键是否被按下。

```
typedef union
{
  struct
  {
    uint8_t R1 : 1;
    uint8_t L1 : 1;
    uint8_t start : 1;
    uint8_t select : 1;
```

```
            uint8_t R2 : 1;
            uint8_t L2 : 1;
            uint8_t F1 : 1;
            uint8_t F2 : 1;
            uint8_t A : 1;
            uint8_t B : 1;
            uint8_t X : 1;
            uint8_t Y : 1;
            uint8_t up : 1;
            uint8_t right : 1;
            uint8_t down : 1;
            uint8_t left : 1;
        } components;
        uint16_t value;
    } xKeySwitchUnion;
```

在 main 函数中会创建一个 joystick_dds_model dds 模型，并订阅"rt/wirelesscontroller"节点的消息，当遥控器数值发生改变时，程序会进入 JoystickHandler() 回调函数中，输出遥控器的数值。

完整例程如下：

```
#include <unitree/robot/channel/channel_subscriber.hpp>

#include <unitree/idl/go2/WirelessController_.hpp>

#include <unitree/robot/client/client.hpp>

#define TOPIC_JOYSTICK "rt/wirelesscontroller"

using namespace unitree::common;

// 遥控器键值联合体

typedef union

{

  struct

  {

    uint8_t R1 : 1;

    uint8_t L1 : 1;

    uint8_t start : 1;

    uint8_t select : 1;

    uint8_t R2 : 1;

    uint8_t L2 : 1;

    uint8_t F1 : 1;
```

```
            uint8_t F2 : 1;
            uint8_t A : 1;
            uint8_t B : 1;
            uint8_t X : 1;
            uint8_t Y : 1;
            uint8_t up : 1;
            uint8_t right : 1;
            uint8_t down : 1;
            uint8_t left : 1;
        } components;
        uint16_t value;
    } xKeySwitchUnion;
    // 获取运动状态的回调函数
    void JoystickHandler(const void *message)
    {
        unitree_go::msg::dds_::WirelessController_ joystick =
*(unitree_go::msg::dds_::WirelessController_ *)message;
        // 遥控器原始数据
```

```
    std::cout << "lx: " << joystick.lx() << std::endl       // 左摇杆 x
              << "ly: " << joystick.ly() << std::endl       // 左摇杆 y
              << "rx: " << joystick.rx() << std::endl       // 右摇杆 x
              << "ry: " << joystick.ry() << std::endl       // 右摇杆 y
              << "keys: " << joystick.keys() << std::endl // 键值
              << "------" << std::endl;
    xKeySwitchUnion key;
    key.value = joystick.keys();
    // 判断某个按键是否被按下
    if ((int)key.components.A == 1)
    {
        std::cout << "The key A is pressed " << std::endl;
    }
}
int main(int argc, char **argv)
{
    // 初始化 sdk 接口
    if (argc < 2)
```

```
        {
            std::cout << "Usage: " << argv[0] << " networkInterface" <<
std::endl;
            exit(-1);
        }
        unitree::robot::ChannelFactory::Instance()->Init(0,          argv[1]);
unitree::robot::ChannelSubscriberPtr<unitree_go::msg::dds_::WirelessC
ontroller_> joystick_suber;
        joystick_suber.reset(new
unitree::robot::ChannelSubscriber<unitree_go::msg::dds_::WirelessCont
roller_>(TOPIC_JOYSTICK));
        joystick_suber->InitChannel(JoystickHandler, 1);
        while (1)
        {
            usleep(20000);
        }
        return 0;
    }
```

2.2、 Python 例程实现

```
import time
import sys
from unitree_sdk2py.core.channel import ChannelSubscriber, ChannelFactoryInitialize
from unitree_sdk2py.idl.default import unitree_go_msg_dds__WirelessController_
from unitree_sdk2py.idl.unitree_go.msg.dds_ import WirelessController_
key_state = [
    ["R1", 0],
    ["L1", 0],
    ["start", 0],
    ["select", 0],
    ["R2", 0],
    ["L2", 0],
    ["F1", 0],
```

```
        ["F2", 0],
        ["A", 0],
        ["B", 0],
        ["X", 0],
        ["Y", 0],
        ["up", 0],
        ["right", 0],
        ["down", 0],
        ["left", 0],
    ]
    def WirelessControllerHandler(msg: WirelessController_):
        global key_state
        print("lx: ", msg.lx)
        print("lx: ", msg.ly)
        print("lx: ", msg.rx)
        print("lx: ", msg.ry)
        print("keys: ", msg.keys)
        #Update key state
```

```
        for i in range(16):
            key_state[i][1] = (msg.keys & (1 << i)) >> i
        print(key_state)
    if __name__ == "__main__":
        if len(sys.argv)>1:
            ChannelFactoryInitialize(0, sys.argv[1])
        else:
            ChannelFactoryInitialize(0)
        sub = ChannelSubscriber("rt/wirelesscontroller",
WirelessController_)
        sub.Init(WirelessControllerHandler, 10)


        while True:
            time.sleep(10.0)
```

3、 遥控器高级封装

上述例程只能获取遥控器按键的“按下”/“松开”两种状态，无法反映当下是否处于按下瞬间，并且读取到的摇杆数据为原始数值，不包含死区、滤波等。若想要解决以上问题，必须对遥控器数据进行

高级封装。

3.1、 按钮类

首先定义一个按键状态类型如下。pressed 表示该按钮当前是否处于按下状态，on_press / on_release 表示按钮当前是否恰好处于按下或松开状态。

```
class Button
    {
    public:
        Button() {}
        void update(bool state)
        {
            on_press = state ? state != pressed : false;
            on_release = state ? false : state != pressed;
            pressed = state;
        }
        bool pressed = false;
        bool on_press = false;
        bool on_release = false;
```

```
};
```

3.2、 高级遥控器类

随后定义新的遥控器类型，根据遥控器状态消息更新遥控器按键、摇杆状态。

```
// full gamepad
 class Gamepad
 {
 public:
     Gamepad() {}
     void Update(unitree_go::msg::dds_::WirelessController_
&key_msg)
     {
         // update stick values with smooth and deadzone
         lx = lx * (1 - smooth) + (std::fabs(key_msg.lx()) <
dead_zone ? 0.0 : key_msg.lx()) * smooth;
         rx = rx * (1 - smooth) + (std::fabs(key_msg.rx()) <
dead_zone ? 0.0 : key_msg.rx()) * smooth;
         ry = ry * (1 - smooth) + (std::fabs(key_msg.ry()) <
```

```
dead_zone ? 0.0 : key_msg.ry()) * smooth;

                ly = ly * (1 - smooth) + (std::fabs(key_msg.ly()) <

dead_zone ? 0.0 : key_msg.ly()) * smooth;

                // update button states

                key.value = key_msg.keys();

                R1.update(key.components.R1);

                L1.update(key.components.L1);

                start.update(key.components.start);

                select.update(key.components.select);

                R2.update(key.components.R2);

                L2.update(key.components.L2);

                F1.update(key.components.F1);

                F2.update(key.components.F2);

                A.update(key.components.A);

                B.update(key.components.B);

                X.update(key.components.X);

                Y.update(key.components.Y);

                up.update(key.components.up);
```

```
        right.update(key.components.right);
        down.update(key.components.down);
        left.update(key.components.left);
    }
    float smooth = 0.03;
    float dead_zone = 0.01;
    float lx = 0.;
    float rx = 0.;
    float ry = 0.;
    float ly = 0.;
    Button R1;
    Button L1;
    Button start;
    Button select;
    Button R2;
    Button L2;
    Button F1;
    Button F2;
```

```
        Button A;
        Button B;
        Button X;
        Button Y;
        Button up;
        Button right;
        Button down;
        Button left;
    private:
        xKeySwitchUnion key;
    };
```

3.3、 业务逻辑实现

在编程实践中，可以使用两个线程，一个线程监听并存储遥控器消息，另一个线程定时执行，更新遥控器状态并执行控制逻辑。以计算 A 键被按下的次数为例，两个线程的回调函数如下：

```
class GamepadExample
{
public:
```

```
        // ...
        // 消息监听回调
        void MessageHandler(const void *message)
        {
            std::lock_guard<std::mutex> lock(joystick_mutex);
            joystick_msg = *(unitree_go::msg::dds_::WirelessController_
*)message;
        }
        // 定时控制回调
        void Step()
        {
            {
                std::lock_guard<std::mutex> lock(joystick_mutex);
                gamepad.Update(joystick_msg);
            }
            // some operations
            if (gamepad.A.on_press)
            {
```

```
                press_count += 1;
            }
        }
        // ...
    protected:
        // ...
        unitree_go::msg::dds_::WirelessController_ joystick_msg;
        Gamepad gamepad;
        std::mutex joystick_mutex;
        int press_count = 0;
    };
```

计算 A 键被按下的次数的例程如下。

```
#include <mutex>
#include "unitree/idl/go2/WirelessController_.hpp"
#include "unitree/robot/channel/channel_subscriber.hpp"
#include "gamepad_test.hpp"
#define TOPIC_JOYSTICK "rt/wirelesscontroller"
using namespace unitree::common;
```

```
using namespace unitree::robot;
class GamepadExample
{
public:
    GamepadExample() {}
    // setup dds model
    void InitDdsModel(const std::string &networkInterface = "")
    {
        ChannelFactory::Instance()->Init(0, networkInterface);
        joystick_subscriber.reset(new
ChannelSubscriber<unitree_go::msg::dds_::WirelessController_>(TOPIC
_JOYSTICK));

joystick_subscriber->InitChannel(std::bind(&GamepadExample::Messa
geHandler, this, std::placeholders::_1), 1);
    }
    // set gamepad dead_zone parameter
    void SetGamepadDeadZone(float deadzone)
```

```
        {
            gamepad.dead_zone = deadzone;
        }
        // set gamepad smooth parameter
        void setGamepadSmooth(float smooth)
        {
            gamepad.smooth = smooth;
        }
        // callback function to save joystick message
        void MessageHandler(const void *message)
        {
            std::lock_guard<std::mutex> lock(joystick_mutex);
            joystick_msg = *(unitree_go::msg::dds_::WirelessController_
*)message;
        }
        // work thread
        void Step()
        {
```

```
            {
                std::lock_guard<std::mutex> lock(joystick_mutex);
                gamepad.Update(joystick_msg);
            }
            // some operations
            if (gamepad.A.on_press)
            {
                press_count += 1;
            }
            // print gamepad state
            std::cout << "lx: " << gamepad.lx << std::endl
                      << "A: pressed: " << gamepad.A.pressed
                      << "; on_press: " << gamepad.A.on_press
                      << "; on_release: " << gamepad.A.on_release
                      << std::endl << "press count: " << press_count
                      <<                    std::endl                    <<
"============================" << std::endl;
        }
```

```
        // start the work thread
        void Start()
        {
            control_thread_ptr = CreateRecurrentThreadEx("nn_ctrl",
UT_CPU_ID_NONE, 40000, &GamepadExample::Step, this);
        }
    protected:
ChannelSubscriberPtr<unitree_go::msg::dds_::WirelessController_>
joystick_subscriber;
        unitree_go::msg::dds_::WirelessController_ joystick_msg;
        Gamepad gamepad;
        ThreadPtr control_thread_ptr;
        std::mutex joystick_mutex;
        int press_count = 0;
    };
    int main()
    {
        // create example object
```

```
    GamepadExample example;
    // set gamepad params
    example.setGamepadSmooth(0.2);
    example.SetGamepadDeadZone(0.5);
    // start program
    example.InitDdsModel();
    example.Start();
    while (1)
    {
        usleep(20000);
    }
    return 0;
}
```

4、 按键功能开发

因为控制权限及安全方面的限制，遥控器上已有的按键功能无法屏蔽，我们需要选择未有固定映射的按键进行开发。

4.1、 新映射开发思路一

我们可以让单键直接绑定某个高级运动，比如单按“L1”键，控

制 Go2 机器人执行坐下的动作。

我们需要先添加一个机器人运动控制类（SportModeTest），用于控制机器人执行各种高级运动控制。

```
class SportModeTest:
    def __init__(self) -> None:
        # Time count
        self.t = 0
        self.dt = 0.01
        # Initial poition and yaw
        self.px0 = 0
        self.py0 = 0
        self.yaw0 = 0
        self.client = SportClient()
        self.client.SetTimeout(10.0)
        self.client.Init()
    def GetInitState(self, robot_state: SportModeState_):
        self.px0 = robot_state.position[0]
        self.py0 = robot_state.position[1]
```

```
        self.yaw0 = robot_state.imu_state.rpy[2]

    def Forward(self):

        elapsed_time = 1

        for i in range(int(elapsed_time / self.dt)):

            self.client.Move(0.3, 0, 0)   # vx, vy vyaw

            time.sleep(self.dt)

        self.client.StopMove()

    def Backward(self):

        elapsed_time = 1

        for i in range(int(elapsed_time / self.dt)):

            self.client.Move(-0.3, 0, 0)   # vx, vy vyaw

            time.sleep(self.dt)

        self.client.StopMove()

    def Left(self):

        elapsed_time = 1

        for i in range(int(elapsed_time / self.dt)):

            self.client.Move(0, 0.3, 0)   # vx, vy vyaw

            time.sleep(self.dt)
```

```
        self.client.StopMove()
    def Right(self):
        elapsed_time = 1
        for i in range(int(elapsed_time / self.dt)):
            self.client.Move(0, -0.3, 0)   # vx, vy vyaw
            time.sleep(self.dt)
        self.client.StopMove()
    #close SwithJoystick
    def closeJoystick(self):
        self.client.SwitchJoystick(False)
    #open SwithJoystick
    def openJoystick(self):
        self.client.SwitchJoystick(True)
    def stop(self):
        self.client.StopMove()
```

在按键事件回调函数中侦测“L1”键的按下情况，并添加“坐下”这个新的功能映射。

```
if(int(msg.keys) == 32):
```

```
        print("L2 is pressing")
        robot.sit()
```

4.2、 新映射开发思路二

我们也可以截获“up”、“down”两个键的按键信息，用来控制 Go2 机器人头部的灯光亮度。当按下“up”键时，灯光亮度依次增加；按下“down”键时，灯光亮度依次减弱。

我们需要先初始化灯光亮度控制类（VuiClient），用于执行灯光亮度控制。

```
        vui = VuiClient()
        vui.SetTimeout(3.0)
        vui.Init()
        lightLevel = 0
```

在按键事件回调函数中侦测“up”键和“down”键的按下情况，添加相应的功能映射。

```
    elif(key_state[12][1] == 1):
        print("up is pressing")
        if(lightLevel==10):
            lightLevel = 0
```

```
            else:
                lightLevel = lightLevel+1
            vui.SetBrightness(lightLevel)
        elif(key_state[14][1] == 1):
            print("down is pressing")
            if(lightLevel==0):
                lightLevel = 10
            else:
                lightLevel = lightLevel-1
            vui.SetBrightness(lightLevel)
```

5、 项目总结

通过本项目的深入研究，我们掌握了 Go2 机器人遥控器的数据传输原理和获取机制，并实现了遥控器高级封装应用和自定义功能开发。接下来，我们将进入避障数据获取项目，进一步探索和理解 Go2 机器人的视觉性能。